

Lab 05 Especial - Trevisan - 277200

Bruce Trevisan

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(ggplot2)
library(dplyr)
library(leaflet)
library(ggcal)
library(readr)
```

```
# Consulta de nome das colunas dos arquivos
```

```
colunas_flights <- colnames(read_csv("flights.csv.zip", n_max = 0, show_col_types = FALSE))
```

```
print(colunas_flights)
```

[1] "YEAR"	"MONTH"	"DAY"
[4] "DAY_OF_WEEK"	"AIRLINE"	"FLIGHT_NUMBER"
[7] "TAIL_NUMBER"	"ORIGIN_AIRPORT"	"DESTINATION_AIRPORT"
[10] "SCHEDULED_DEPARTURE"	"DEPARTURE_TIME"	"DEPARTURE_DELAY"
[13] "TAXI_OUT"	"WHEELS_OFF"	"SCHEDULED_TIME"
[16] "ELAPSED_TIME"	"AIR_TIME"	"DISTANCE"

```

[19] "WHEELS_ON"           "TAXI_IN"           "SCHEDULED_ARRIVAL"
[22] "ARRIVAL_TIME"        "ARRIVAL_DELAY"     "DIVERTED"
[25] "CANCELLED"           "CANCELLATION_REASON" "AIR_SYSTEM_DELAY"
[28] "SECURITY_DELAY"      "AIRLINE_DELAY"      "LATE_AIRCRAFT_DELAY"
[31] "WEATHER_DELAY"

```

```

# Para mostrar nomes de linha de voos diferentes do enunciado
primeiros_nomes <- read_csv(
  file = "flights.csv.zip",
  col_select = "TAIL_NUMBER",
  n_max = 5,
  show_col_types = FALSE
)

print(primeiros_nomes)

```

```

# A tibble: 5 x 1
  TAIL_NUMBER
  <chr>
1 N407AS
2 N3KUAA
3 N171US
4 N3HYAA
5 N527AS

```

```

# Tanto que a linha de voo escolhida foi a 5ª dessa lista

```

```

# Leitura de arquivo
analisa_aeronave <- function(tail_number, arquivo, arquivo_airports = "airports.csv") {

  dados_aeroportos <- read_csv(
    arquivo_airports,
    show_col_types = FALSE
  ) %>%
  select(
    codigo_aeroporto = IATA_CODE,
    nome_aeroporto = AIRPORT,
    cidade = CITY,
    latitude = LATITUDE,
    longitude = LONGITUDE
  ) %>%

```

```

    filter(!is.na(latitude), !is.na(longitude))

# Leitura do flights em chunks
callback_filtro_aeronave <- function(chunk, pos) {
  chunk %>% filter(TAIL_NUMBER == tail_number)
}

tabela_voos_filtrada <- read_csv_chunked(
  file = arquivo,
  callback = DataFrameCallback$new(callback_filtro_aeronave),
  chunk_size = 500000,
  show_col_types = FALSE
)

# Validação de carregamento
if (nrow(tabela_voos_filtrada) == 0) {
  stop(paste("Nenhum voo encontrado para a aeronave:", tail_number))
}

tabela_trajetos_tidy <- tabela_voos_filtrada %>%
  filter(
    CANCELLED == 0,
    !is.na(DEPARTURE_TIME),
    !is.na(AIR_TIME),
    AIR_TIME > 0
  ) %>%
  mutate(
    horario_saida_formatado = sprintf("%s", DEPARTURE_TIME),
    data_hora_partida = as.POSIXct(
      paste(
        YEAR, MONTH, DAY,
        substr(horario_saida_formatado, 1, 2),
        substr(horario_saida_formatado, 3, 4)
      ),
      format = "%Y %m %d %H %M"
    ),
    velocidade_media_mph = (DISTANCE / AIR_TIME) * 60
  ) %>%
  arrange(data_hora_partida)

# Junção dos dados
tabela_com_coordenadas <- tabela_trajetos_tidy %>%

```

```

inner_join(
  dados_aeroportos,
  by = c("ORIGIN_AIRPORT" = "codigo_aeroporto")
) %>%
rename(
  lat_origem = latitude,
  lng_origem = longitude,
  nome_origem = nome_aeroporto
) %>%
inner_join(
  dados_aeroportos,
  by = c("DESTINATION_AIRPORT" = "codigo_aeroporto")
) %>%
rename(
  lat_destino = latitude,
  lng_destino = longitude,
  nome_destino = nome_aeroporto
)

# Mapa
mapa_trajeto <- leaflet() %>%
  addProviderTiles(providers$CartoDB.Positron)

aeroportos_unicos <- bind_rows(
  tabela_com_coordenadas %>% select(codigo = ORIGIN_AIRPORT, nome = nome_origem, lat = lat, lng = lng),
  tabela_com_coordenadas %>% select(codigo = DESTINATION_AIRPORT, nome = nome_destino, lat = lat, lng = lng)
) %>% distinct(codigo, .keep_all = TRUE)

mapa_trajeto <- mapa_trajeto %>%
  addCircleMarkers(
    data = aeroportos_unicos,
    lng = ~lng,
    lat = ~lat,
    popup = ~paste0("<b>Aeroporto:</b> ", nome, " (", codigo, ")"),
    radius = 4,
    color = "#2c3e50",
    fillOpacity = 0.8
  )

for (i in seq_len(nrow(tabela_com_coordenadas))) {
  voo_atual <- tabela_com_coordenadas[i, ]

```

```

    espessura_linha <- scales::rescale(
      voo_atual$velocidade_media_mph,
      to = c(2, 8),
      from = range(tabela_com_coordenadas$velocidade_media_mph, na.rm = TRUE)
    )

    texto_popup <- str_glue("
    <b>Voo nº:</b> {i} de {nrow(tabela_com_coordenadas)}<br>
    <b>Data:</b> {format(voo_atual$data_hora_partida, '%d/%m/%Y %H:%M')}<br>
    <b>Trecho:</b> {voo_atual$ORIGIN_AIRPORT} {voo_atual$DESTINATION_AIRPORT}<br>
    <b>Distância:</b> {voo_atual$DISTANCE} milhas<br>
    <b>Velocidade Média:</b> {round(voo_atual$velocidade_media_mph, 1)} mph
    ")

    mapa_trajeto <- mapa_trajeto %>%
      addPolylines(
        lng = c(voo_atual$lng_origem, voo_atual$lng_destino),
        lat = c(voo_atual$lat_origem, voo_atual$lat_destino),
        weight = espessura_linha,
        color = "#9D00FF",
        opacity = 0.4,
        popup = texto_popup
      )
  }

  # 6. Retorno
  resultado <- list(
    tabela = tabela_trajetos_tidy,
    mapa = mapa_trajeto
  )

  return(resultado)
}

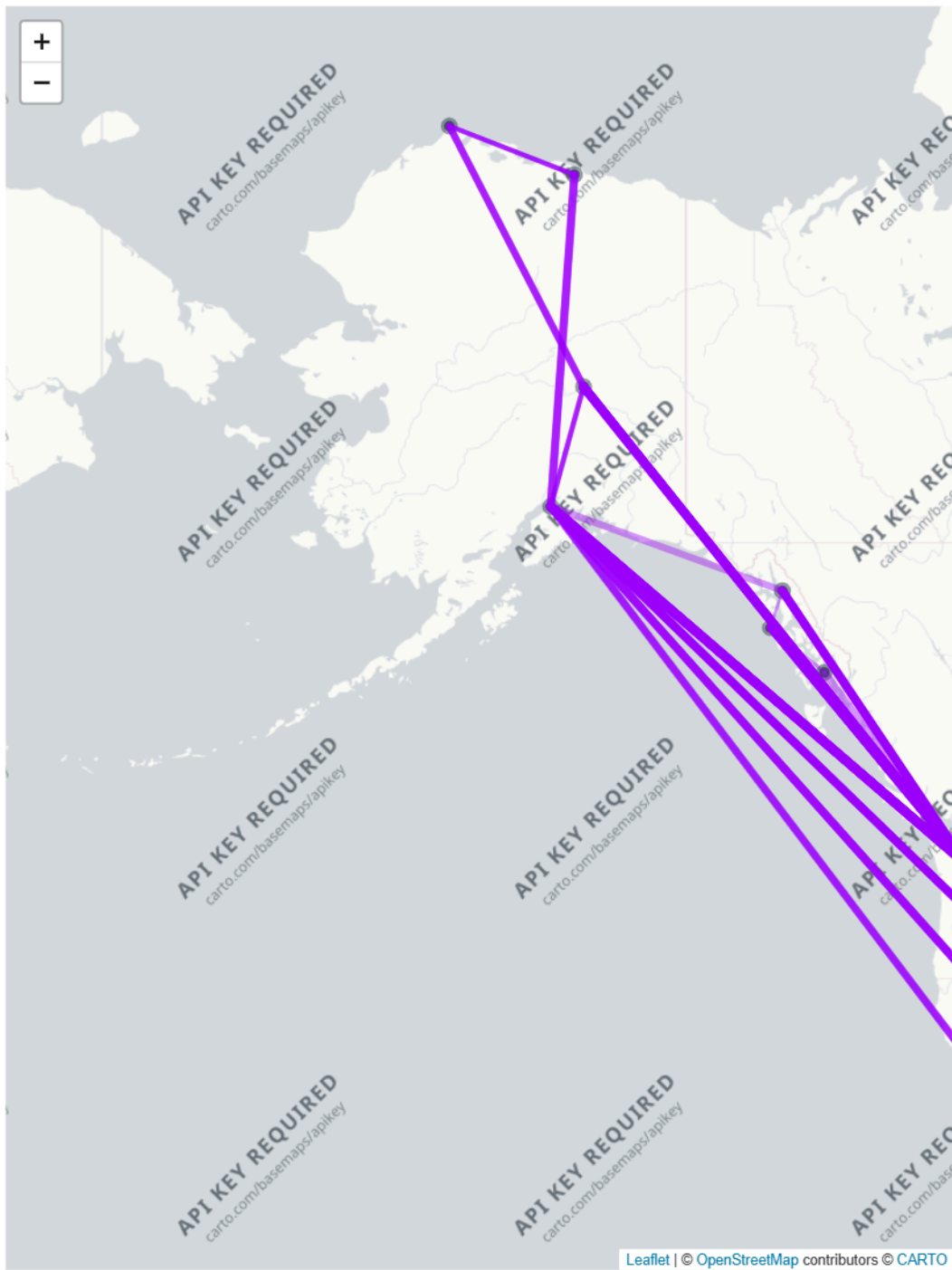
```

```

# Mostrar o mapa
resultado <- analisa_aeronave(
  tail_number = "N527AS",
  arquivo = "flights.csv.zip",
  arquivo_airports = "airports.csv"
)

resultado$mapa

```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 23:32:54 -03"
```